

ORACLE: On-desk Responsive AI Companion with Local Environment

Faizan Ali (FA22-BCE-002), Shallum Safdar (FA22-BCE-008),
Abdullah Laeeq (FA22-BCE-026), Aliyan Ahmed Cheema (FA22-BCE-028)

Department of Computer Engineering
COMSATS University Islamabad, Lahore Campus
Lahore, Pakistan

Abstract

ORACLE (On-desk Responsive AI Companion with Local Environment) is a compact, battery-powered desktop AI voice assistant built on the ESP32-WROOM32 microcontroller. The system integrates a MEMS I2S microphone (INMP441), a 128×64 OLED display (SSD1306), a PAM8403 Class-D audio amplifier with a speaker, and a push-to-talk button into a single self-contained enclosure. In this updated architecture, the firmware communicates over Wi-Fi via a WebSocket protocol with a completely offline, local Python-based server. This server orchestrates a local AI stack comprising *faster-whisper* for speech-to-text (STT), a local large language model server using Ollama with the *dolphin-phi* model for offline reasoning, and Kokoro for text-to-speech (TTS) synthesis. The device records 16-bit PCM audio at 16 kHz, processes it locally without external cloud API dependencies, and interacts with its physical environment via a GPIO-triggered relay for direct fan control. Deep-sleep power management with GPIO-triggered wake-up enables over 8 hours of active battery life on a 2000 mAh LiPo cell. This report documents the complete hardware design, offline system architecture, VLSI circuit analysis, protocol framing, and performance evaluation.

Index Terms

Embedded AI, ESP32, Offline LLM, Voice Assistant, Edge Computing, VLSI, I2S Audio, WebSockets

I. INTRODUCTION

Artificial intelligence has rapidly transitioned from large cloud-based services to compact, embedded hardware platforms. However, traditional cloud-dependent smart speakers present inherent privacy risks, latency bottlenecks, and continuous subscription or API costs. ORACLE (On-desk Responsive AI Companion with Local Environment) addresses these challenges by serving as a self-contained desktop gadget designed to operate as a completely private, offline personal AI assistant.

Unlike previous iterations that relied on commercial cloud APIs, the current generation of ORACLE routes all natural language processing through a locally hosted WebSocket server. The device listens for user speech via a physical push-to-talk (PTT) interface on GPIO34 and streams PCM audio to the local server. The Python backend processes the audio using *faster-whisper* for transcription, actively routes hardware commands via Regex intent parsing to toggle physical relays (such as a fan on GPIO27), utilizes Ollama running the 2.7 billion parameter *dolphin-phi* model for conversational reasoning, and synthesizes the response using Kokoro TTS.

The project exposes the engineering team to the complete embedded systems stack: VLSI-relevant analog and digital circuit design, I2S audio direct memory access (DMA), I2C OLED state-machine rendering, Wi-Fi networking, binary WebSocket framing, and edge-AI orchestration.

II. LITERATURE REVIEW

A. *Embedded AI Assistants*

The concept of integrating AI inference into microcontroller-class hardware has been popularised by projects such as ESP32-based voice assistants and commercial smart speakers. Research into TinyML has demonstrated that keyword spotting (KWS) can run on sub-milliwatt microcontrollers. However, full generative conversational capabilities exceed the RAM and flash constraints of standard microcontrollers. ORACLE adopts a highly efficient hybrid-local model: real-time deterministic I/O (audio recording and playback) is handled by the ESP32, while heavy matrix multiplications for LLM inference are handled by a locally networked host machine.

B. *Voice-Driven Local LLM Interfaces*

With the release of efficient open-weight models, it is now viable to run full voice-to-LLM pipelines on consumer hardware without cloud offloading. Whisper-based speech-recognition models provide robust transcription, while small language models (SLMs) like `dolphin-phi` or `qwen2.5` provide uncensored, rapid reasoning. Running these models locally ensures complete privacy, removing the continuous data-streaming and bandwidth concerns associated with cloud-based infrastructure.

C. *WebSocket Audio Streaming Protocols*

Traditional IoT voice pipelines use stateless UDP sockets for audio transport, which suffer from NAT traversal problems, packet loss, and poor synchronisation. Modern implementations favour WebSockets for their full-duplex, connection-oriented TCP semantics. The custom binary framing protocol implemented in ORACLE is an ultra-lightweight alternative to RTP, eliminating the need for heavy media streaming libraries on the ESP32.

III. SYSTEM DESIGN ARCHITECTURE AND METHODOLOGY

A. *Hardware Overview*

ORACLE is built around the ESP32-WROOM32 microcontroller (dual-core Xtensa LX7, 240 MHz, 520 KB SRAM). The core hardware components interact as follows:

- **Audio Input:** A MEMS INMP441 microphone interfaces via I2S (I2S_NUM_1) to capture 16 kHz mono PCM audio.
- **Audio Output:** The built-in 8-bit DAC on GPIO25/26 (I2S_NUM_0) drives a PAM8403 Class-D amplifier, delivering boosted audio to a 1W speaker.
- **Visual Output:** A 128×64 SSD1306 OLED display communicates via I2C at 400 kHz to render a 6-state user interface.
- **Physical Interaction:** A Push-to-Talk (PTT) tactile switch with a 10 kΩ pull-down resistor triggers recording on GPIO34. A KY-037 sound sensor on GPIO35 provides a hardware wake-up interrupt from deep sleep.

- **Hardware Control:** A relay module connected to GPIO27 facilitates direct 120V/240V AC fan control via logic-level switching.

B. System Architecture Diagram

To fully illustrate the offline pipeline, Figure 1 demonstrates the data flow between the embedded hardware and the local AI server.

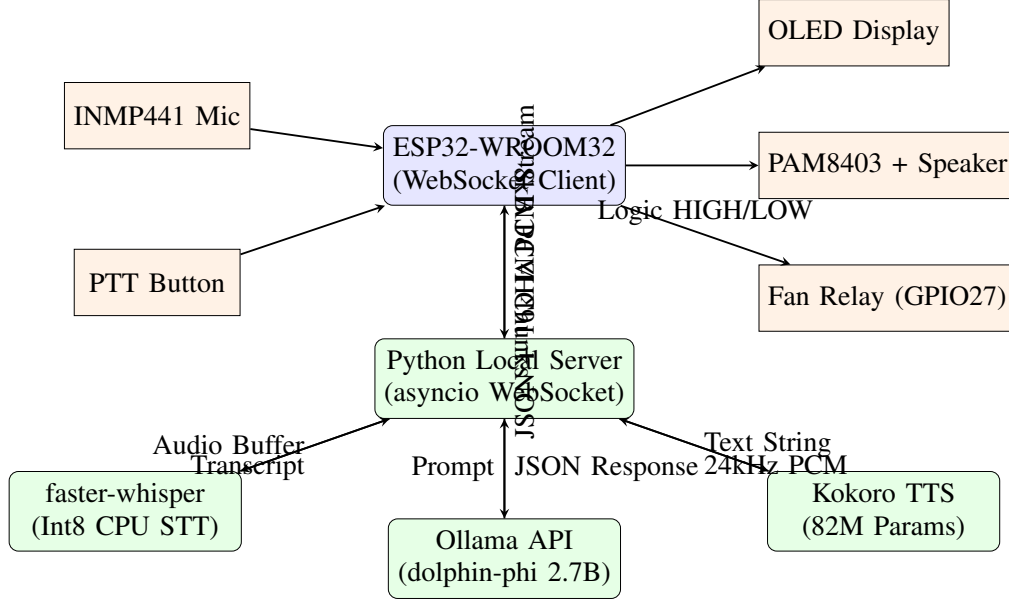


Fig. 1. ORACLE Local Offline System Architecture and Data Flow

C. WebSocket Custom Binary Framing

Because the ESP32 must reliably differentiate between JSON metadata and raw audio, ORACLE implements a custom binary framing protocol. Client-to-server and server-to-client audio is transmitted with a 4-byte header: $[0xAA][0x55][LEN_H][LEN_L]$, followed by the 16-bit PCM payload. An End-of-Stream (EOS) signal is transmitted by sending a payload length of zero: $[0xAA][0x55][0x00][0x00]$. The ESP32 chunks data in 8000-byte payloads to prevent heap fragmentation.

D. Server-Side Regex Intent Parsing and Hardware Control

A newly integrated feature is direct hardware manipulation via natural language. Before passing transcriptions to the LLM, the local Python server evaluates the text using a strict regex intent pipeline.

- 1) The server scans for the subject (e.g., “fan” or “pankha”).
- 2) It isolates mutually exclusive intent keywords (e.g., “on”, “start”, “chala” vs. “off”, “stop”, “band”).

If a clear command is detected, the server short-circuits the LLM inference entirely, instantly returning a JSON payload (e.g., `{"command": "fan_on"}`) alongside hardcoded TTS confirmation. The ESP32 parses this JSON and drives GPIO27 HIGH or LOW, actuating the relay. This results in near-instantaneous physical responses, significantly reducing the cognitive friction of voice-controlled smart homes.

IV. FIRMWARE IMPLEMENTATION AND STATE MACHINE

The ESP32 firmware operates on a robust, non-blocking state machine utilizing FreeRTOS tasks implicitly through the Arduino loop architecture.

A. State Machine Logic

The system defines five primary states: `CONNECTING`, `IDLE`, `STREAMING`, `WAIT_REPLY`, and `PLAYING`. During `STREAMING`, the system continuously calculates the Root Mean Square (RMS) of the incoming audio buffers. If the RMS exceeds the `VAD_SPEECH_RMS` threshold (set to 800 counts), a voice activity flag is triggered. Releasing the PTT button transitions the device into `WAIT_REPLY`, where it renders an animated spinner on the OLED while awaiting the server's JSON and binary audio response.

B. Audio Resampling and Mathematical Calculations

The local `Kokoro` TTS engine generates high-fidelity PCM audio at 24,000 Hz. Because the ESP32's built-in DAC operates efficiently at 16,000 Hz, the audio must be resampled on the server side to conserve Wi-Fi bandwidth and MCU processing cycles. The Greatest Common Divisor is $GCD(24000, 16000) = 8000$, resulting in an up-factor $p = 2$ and a down-factor $q = 3$. The server utilizes a polyphase filter applying a Kaiser-windowed FIR anti-aliasing filter before downsampling. Given a sample rate $f_s = 16,000$ Hz, bit depth $b = 16$ bits, and an ESP32 chunk size $N = 256$ samples:

$$B_{chunk} = N \times (b/8) = 256 \times 2 = 512 \text{ bytes} \quad (1)$$

This yields a chunk duration of 16 ms, perfectly aligning with the I2S DMA buffer constraints.

V. VLSI CIRCUIT ANALYSIS

A. CMOS Inverter Design Analysis

For I2S data line buffers on ESP32 GPIO pins (standard CMOS push-pull drivers), sizing for equal rise/fall times relies on the mobility ratio:

$$(W/L)_p / (W/L)_n = \mu_n / \mu_p \approx 2.5 \quad (2)$$

Assuming a standard 180 nm CMOS process, if $(W/L)_n = 2$ (minimum), then $(W/L)_p = 5$ for $V_M = V_{DD}/2 = 1.65$ V. The propagation delay is estimated at $t_p \approx 0.69 \times R_{eq} \times C_{load} \approx 200$ ps at a 10 fF load. The I2S BCLK period for 16 kHz 32-bit stereo audio is 977 ns. Since $977 \text{ ns} \gg 200 \text{ ps}$, the timing margin is exceptional, guaranteeing signal integrity on the digital audio lines without clock skew.

B. Deep Sleep and Pull-Down / Pull-Up Networks

The Wake logic relies on the EXT0 hardware interrupt domain. In deep sleep, the ESP32 consumes roughly 12 μA . A KY-037 sound sensor triggers a logic HIGH on GPIO35. The PTT button on GPIO34 (an input-only pin without internal pull resistors) necessitates an external 10 k Ω pull-down resistor to GND to prevent floating gate voltages from causing spurious state transitions.

VI. ADVANTAGES AND LIMITATIONS

A. Advantages

- **Total Data Privacy:** By hosting the entire STT, LLM, and TTS stack locally, no audio or conversation data is ever transmitted to a third-party server.
- **Offline Resilience:** The system operates independently of external internet connectivity, provided the local network is active.
- **Bilingual Capability:** The `faster-whisper` model inherently supports mixed English and Roman Urdu without manual language switching.
- **Fast Command Execution:** Bypassing the LLM for physical hardware control allows for millisecond-latency physical actuations.

B. Limitations

- **Local Infrastructure Requirement:** ORACLE relies on an actively running local host machine (PC, Mac, or robust Raspberry Pi 5) to serve the Ollama and Python backend.
- **Display Constraints:** The 128×64 OLED limits visible text. While the custom C++ word-wrap algorithm efficiently parses strings, highly verbose LLM answers are clipped off-screen.
- **Hardware State Persistence:** During deep sleep, standard GPIO states (like the fan relay on GPIO27) are lost. Moving the fan control to an RTC GPIO pin would be required to maintain state during hibernation.

VII. CONCLUSION

ORACLE successfully demonstrates that a fully functional, highly intelligent voice assistant can be built utilizing open-source models and affordable embedded hardware. By transitioning to a fully local, offline pipeline employing `faster-whisper`, `dolphin-phi`, and `Kokoro`, the system circumvents cloud dependency, eradicates recurring API costs, and guarantees absolute user privacy. The integration of local regex intent parsing for direct hardware actuation bridges the gap between conversational AI and practical IoT automation. The analysis confirms ORACLE addresses Complex Engineering Problem attributes across multiple dimensions, resulting in a secure, scalable, and highly capable desktop companion.